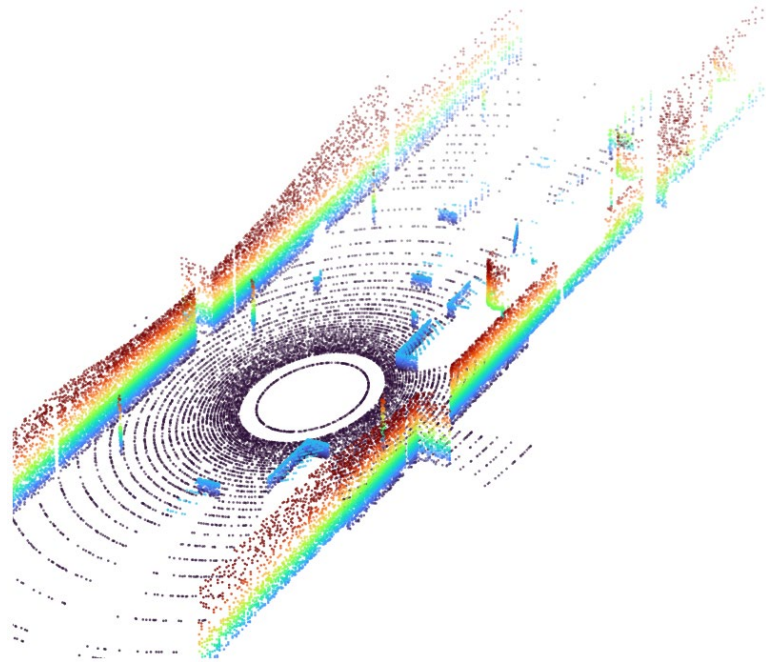


L5 LiDAR Perception



Zonghua Gu | Fall 2026

Roadmap and Learning Outcomes

Part 1

LiDAR Fundamentals

ToF ranging, scanning,
wavelength, weather
Lecture A | ~35 min

Part 2

Point Cloud Data

Frames, calibration,
representations, datasets
Lecture A | ~30 min

Part 3

Classical Perception

Ground removal, clustering,
boxes, ICP, tracking
Lecture B | ~35 min

Part 4

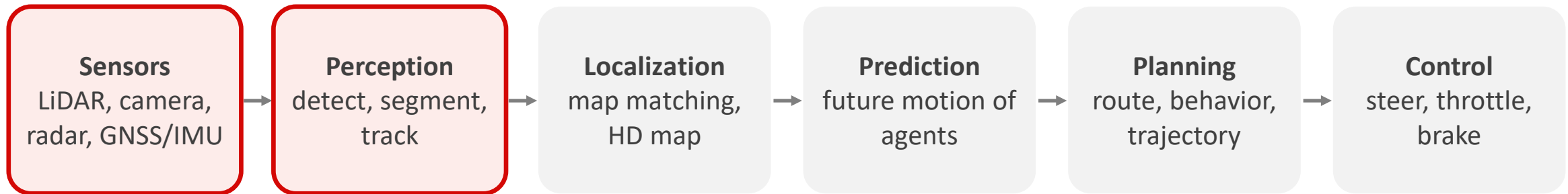
Deep Learning Perception

PointNet, voxels, pillars,
BEV, fusion, metrics
Lecture B | ~40 min

After these two lectures you should be able to:

1. Explain how a LiDAR measures range and why point clouds are sparse, noisy, and range-dependent
2. Transform points between sensor, vehicle, and world frames and compensate motion distortion
3. Build and critique a classical pipeline: ground removal, clustering, box fitting, tracking
4. Compare point-, voxel-, pillar-, and BEV-based 3D detectors and read their metrics

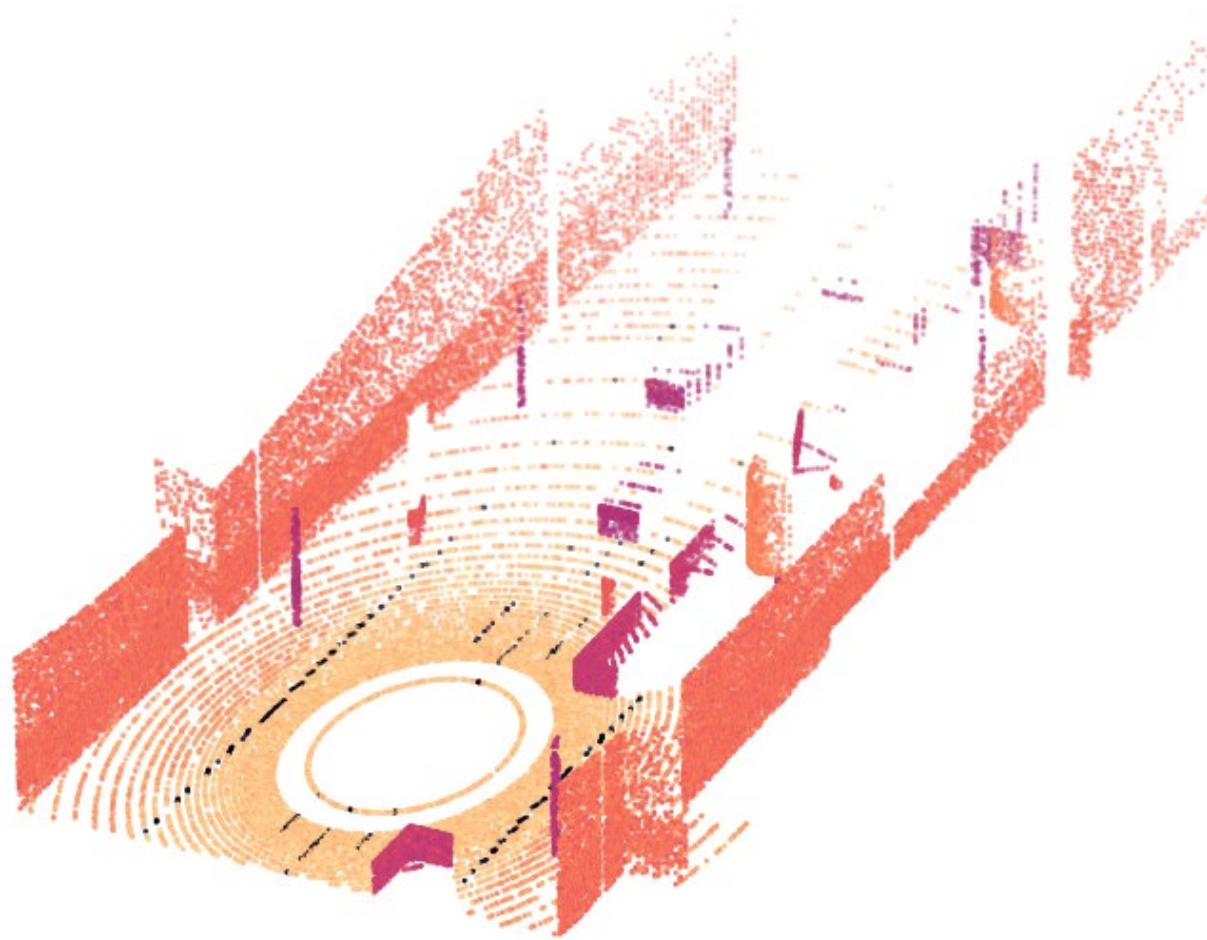
Where LiDAR Fits in the AV Stack



today's focus

- LiDAR gives **direct 3D geometry**: centimeter-level range, independent of ambient light
- Output of perception is a list of **3D boxes with class, heading, velocity, and track ID**
- The same list feeds localization (map matching), prediction, and planning
- Today: how pulses become points, and how points become objects
- Builds on L1.2 (sensor overview) and the 2D detection lecture; links forward to adversarial attacks

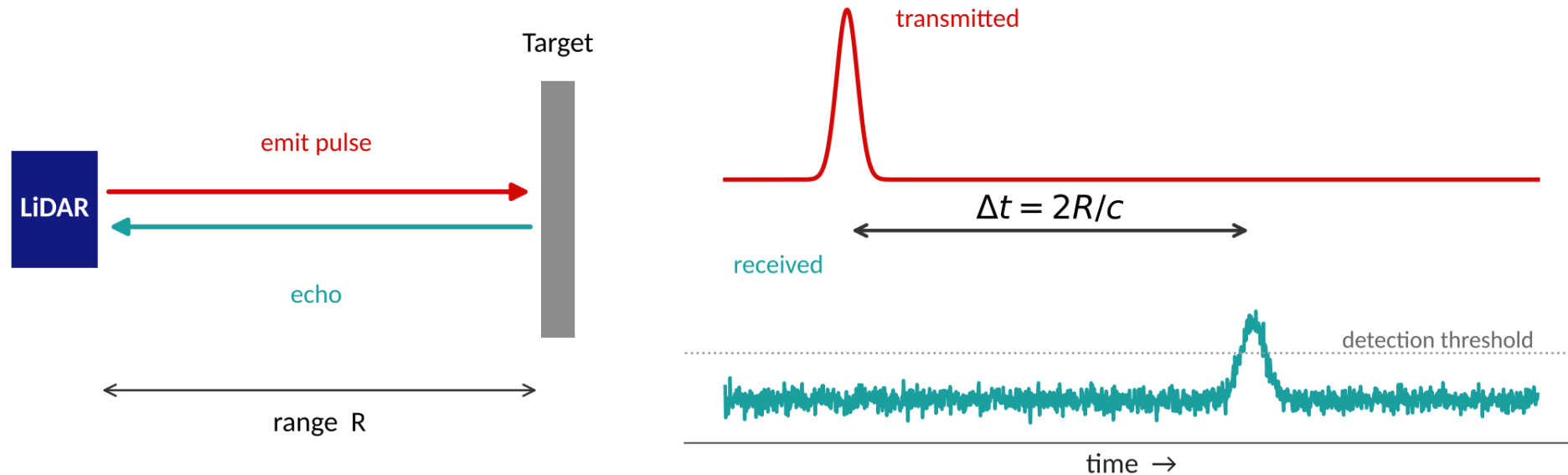
Part 1: LiDAR Fundamentals



Time of flight | coordinates | scanning | wavelength | reflectivity | weather | data rate

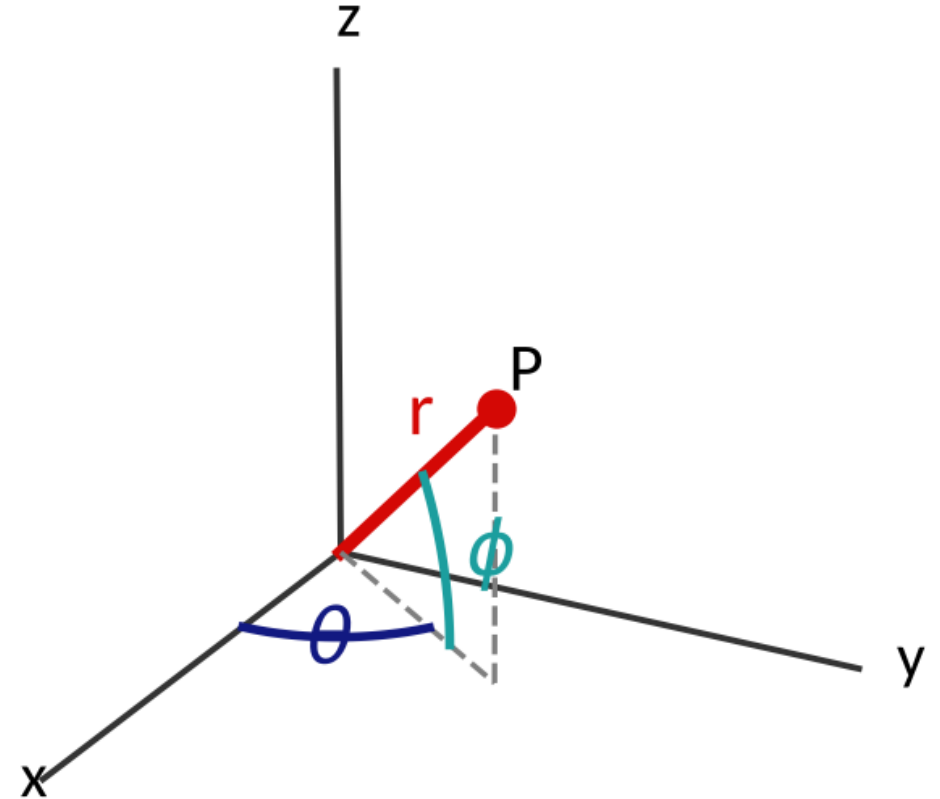
Time-of-Flight Ranging

- A pulse leaves the laser at $t = 0$, hits a surface, and a fraction returns
- Range from the round-trip time: $R = c \cdot \Delta t / 2$ ($c \approx 3 \times 10^8$ m/s)
- Rule of thumb: **1 ns of timing $\approx$ 15 cm of range**; 5 cm accuracy needs ≈ 0.33 ns timing
- A 200 m target returns after $2 \cdot 200 / c \approx 1.33$ μ s
- Per-channel pulse rate is limited by the echo wait: $f \leq c / 2R_{\text{max}}$ (≤ 750 kHz for 200 m)



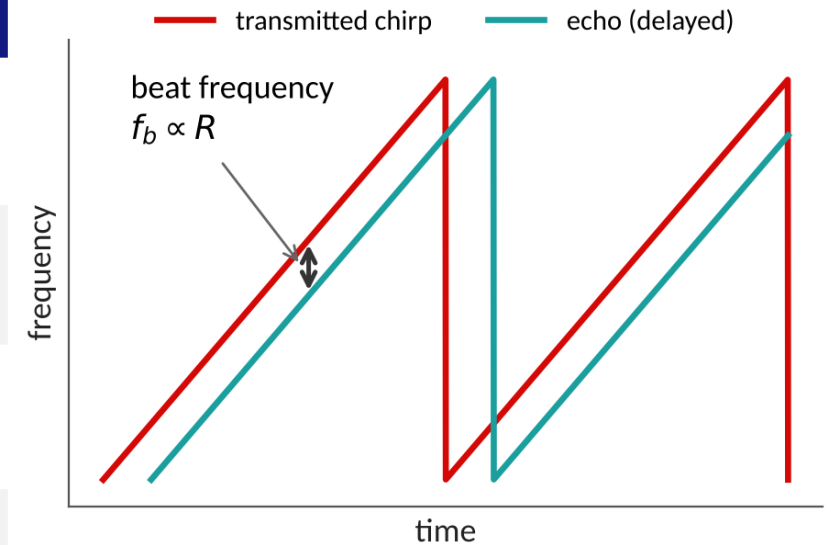
From Measurement to 3D Point

- Raw return: **(r , θ , ϕ , intensity, t)**
- θ = azimuth (encoder / scan angle), ϕ = elevation (fixed per channel)
 - $x = r \cos \phi \cos \theta$
 - $y = r \cos \phi \sin \theta$
 - $z = r \sin \phi$
- Inverse: $r = \sqrt{x^2 + y^2 + z^2}$, $\phi = \arcsin(z / r)$, $\theta = \operatorname{atan2}(y, x)$
- In the sensor frame z points up: the road is near $z \approx -(\text{mounting height})$
- Angular errors matter: 0.1° at 100 m is already ≈ 0.17 m laterally



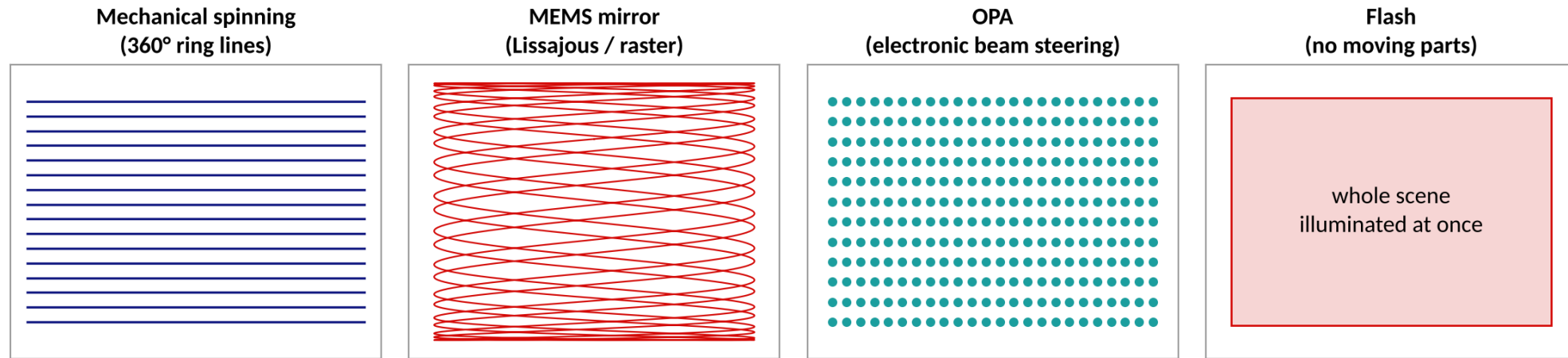
Pulsed ToF vs. FMCW LiDAR

	Pulsed ToF	FMCW
Measures	Round-trip time of a short pulse	Beat frequency of chirp vs. echo
Velocity	Not direct (from frame-to-frame motion)	Direct radial velocity via Doppler
Detection	Incoherent, intensity threshold	Coherent; robust to other LiDARs and sunlight
Wavelength	905–940 nm or 1550 nm	Typically 1550 nm
Maturity	Mass-produced, lower cost	Newer, complex photonics, higher cost
Examples	Hesai, Ouster, Luminar, Waymo	Aeva, Aurora



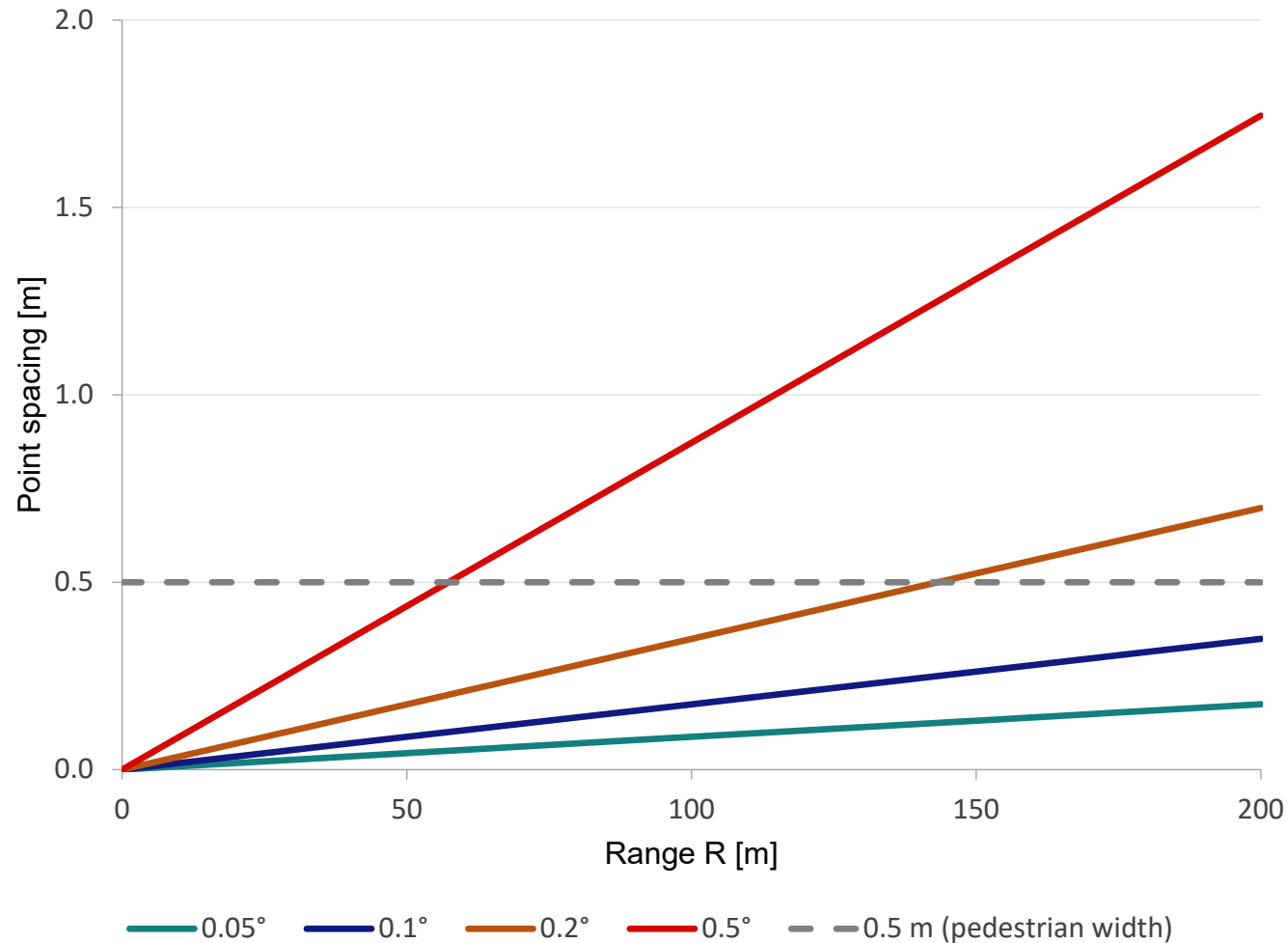
Beat frequency f_b is proportional to range R ; the Doppler shift adds a velocity term.

How the Beam Is Steered



Type	How it works	Strengths	Limits
Mechanical spinning	Rotating assembly sweeps 360°	Full FOV, mature, long range	Cost, size, moving parts
MEMS mirror	Oscillating micro-mirror	Compact, lower cost	Narrow FOV per unit, small aperture
Optical phased array	Electronic beam steering	No moving parts	Immature; limited range and FOV
Flash	Illuminate whole scene at once	Simple, fast frame	Short range, low resolution

Resolution and Point Spacing



Spacing between neighboring points

$$s = R \cdot \tan(\Delta\theta) \approx R \cdot \Delta\theta$$

At $\Delta\theta = 0.2^\circ$:

- 0.17 m at 50 m
- 0.35 m at 100 m
- 0.70 m at 200 m

Beyond ≈ 143 m a 0.5 m pedestrian is hit by less than one beam per scan line

Finer $\Delta\theta$ means more points per second, more compute, more heat

Wavelength Choice: 905 nm vs. 1550 nm

	≈ 905–940 nm	≈ 1550 nm
Source	Semiconductor diode lasers (edge emitters, VCSEL arrays)	Fiber or other 1550 nm lasers
Detector	Silicon APD / SPAD / SiPM (cheap, mature)	InGaAs APD (more expensive)
Eye safety	Light focuses on the retina → pulse energy is limited	Absorbed before the retina → higher pulse energy allowed
Range potential	Limited by eye-safe power; relies on sensitive detectors	Higher power budget helps long range
Atmosphere	Lower water absorption	Stronger water absorption
Cost / volume	Lower cost, widely used	Higher cost, fewer suppliers

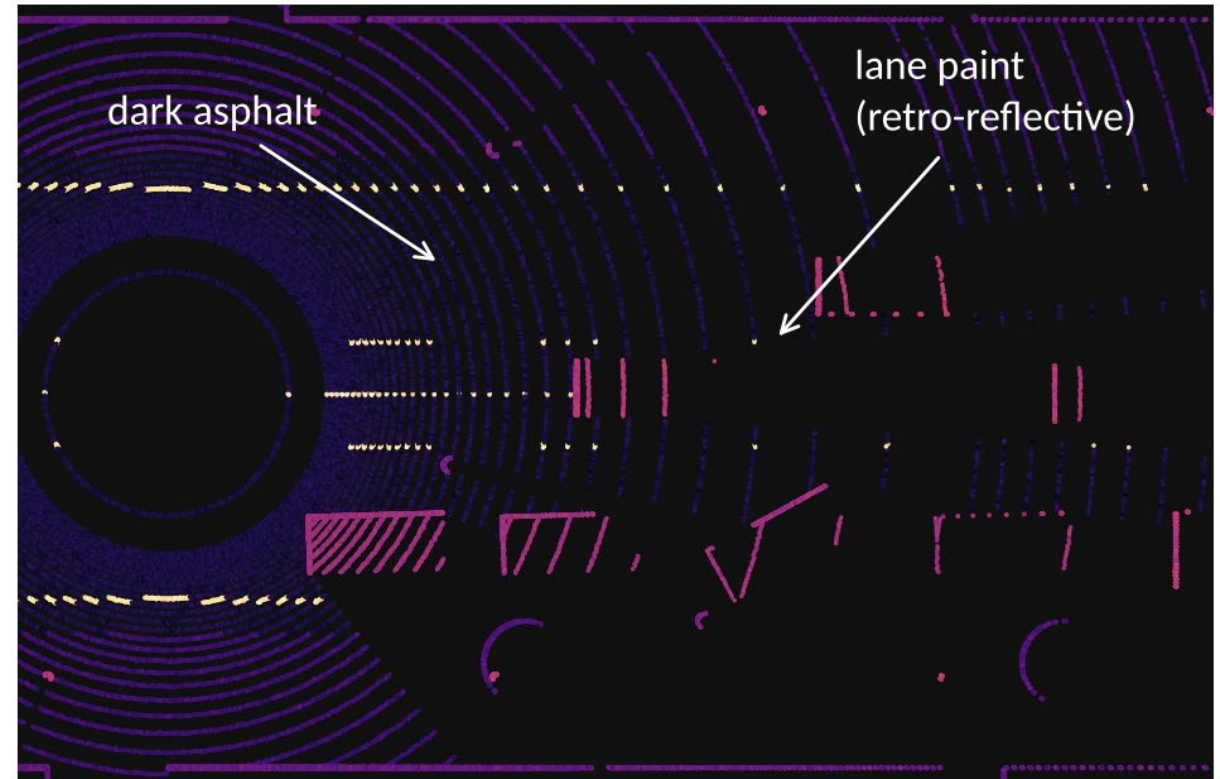
Short-Range vs. Long-Range LiDARs

	Short-range perimeter LiDAR	Long-range 360° LiDAR
Max range	≈ 20 m	≈ 200 m
Azimuth resolution	≈ 0.5°	≈ 0.05°
Listening time (check: $2R/c$)	≈ 0.13 μs	≈ 1.3 μs
SPADs per instantaneous FOV	≈ 10	≈ 1000
Role	Curbs, close pedestrians, blind spots	Highway speed, far vehicles, early warning

Waymo's vehicle combines four short-range perimeter LiDARs, one long-range 360° LiDAR, cameras, and radar; it designs its own pulsed-ToF sensors.

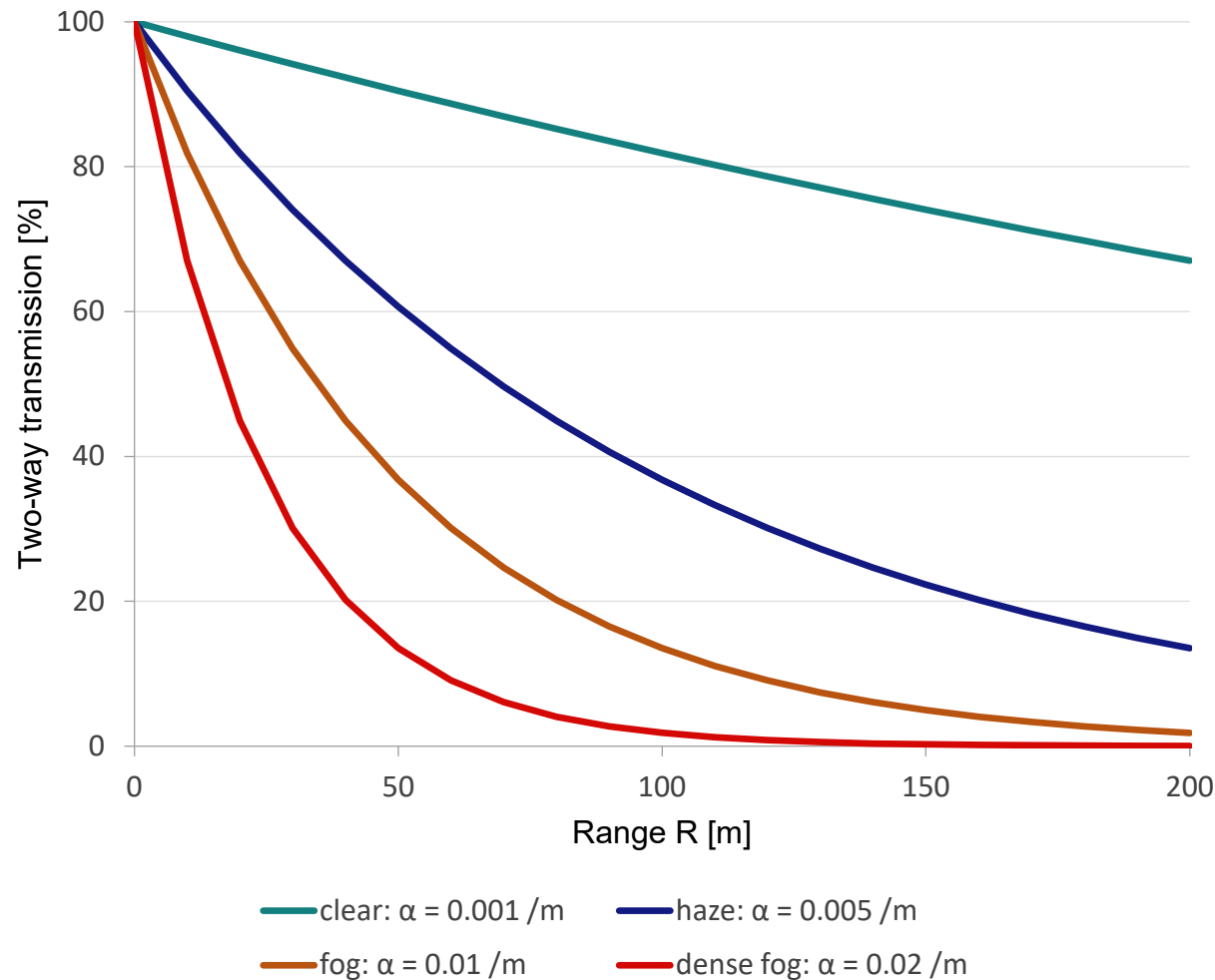
Reflectivity and Intensity

- Return power depends on **reflectivity**, **range** ($\approx 1/R^2$), **incidence angle**, **wavelength**
- Diffuse dark targets (tires, asphalt) return **< 5 %**; retroreflectors (signs, lane paint) are **10–30+ dB** brighter
- Receiver needs a very high dynamic range: weak and strong echoes in the same sweep
- Useful features: lane markings, sign detection, localization against intensity maps
- Caveat: raw intensity is not a calibrated reflectance; normalize by range and angle



Synthetic scene colored by reflectivity (bright = high).

Weather and Window Contamination



- Signal falls as **$\exp(-2\alpha R)$** : the light travels out and back
- Fog, rain, snow add attenuation **and** backscatter clutter (false droplet returns)
- A dirty or wet window scatters light into neighboring channels
- Waymo: detect > 200 m despite weather; use **full-waveform sampling** and **adaptive thresholds**
- Practical filters: isolated low-intensity points, multi-return, cleaning and heating

Typical Failure Modes

Phenomenon	Cause	What you see in the point cloud	Typical mitigation
Specular / glass surfaces	Mirror-like reflection or transmission	Holes, or ghost points behind the surface	Multi-return, camera fusion
Very dark targets	< 5 % of photons return	Sparse or missing points on tires, black cars	Sensitive detectors, longer integration
Multipath / ghost points	Pulse bounces over two surfaces	Phantom points beyond walls and vehicles	Geometric consistency filters
Crosstalk and interference	Other LiDARs, sunlight	Noise points, phantom objects	Pulse coding, timing jitter, FMCW
Retroreflector blooming	Strong echo saturates receiver	Halo around signs and plates	Dynamic range, saturation handling

LiDAR Generates Big Data

2.3 M

points per second
(128 ch × 1,800 az × 10 Hz)

37 MB/s

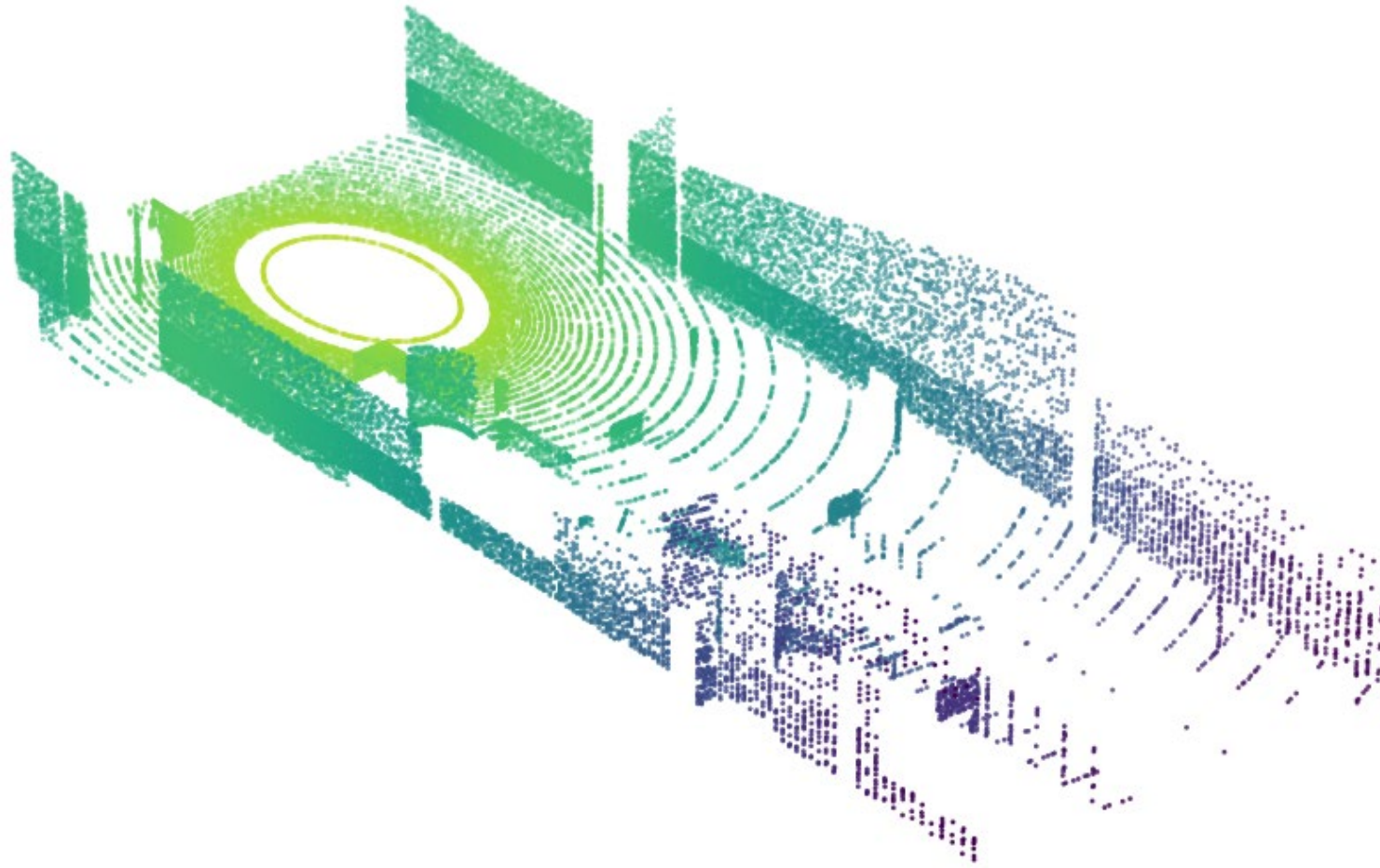
at 16 bytes per point
(x, y, z, intensity as float32)

133 GB/h

from a single sensor,
before compression

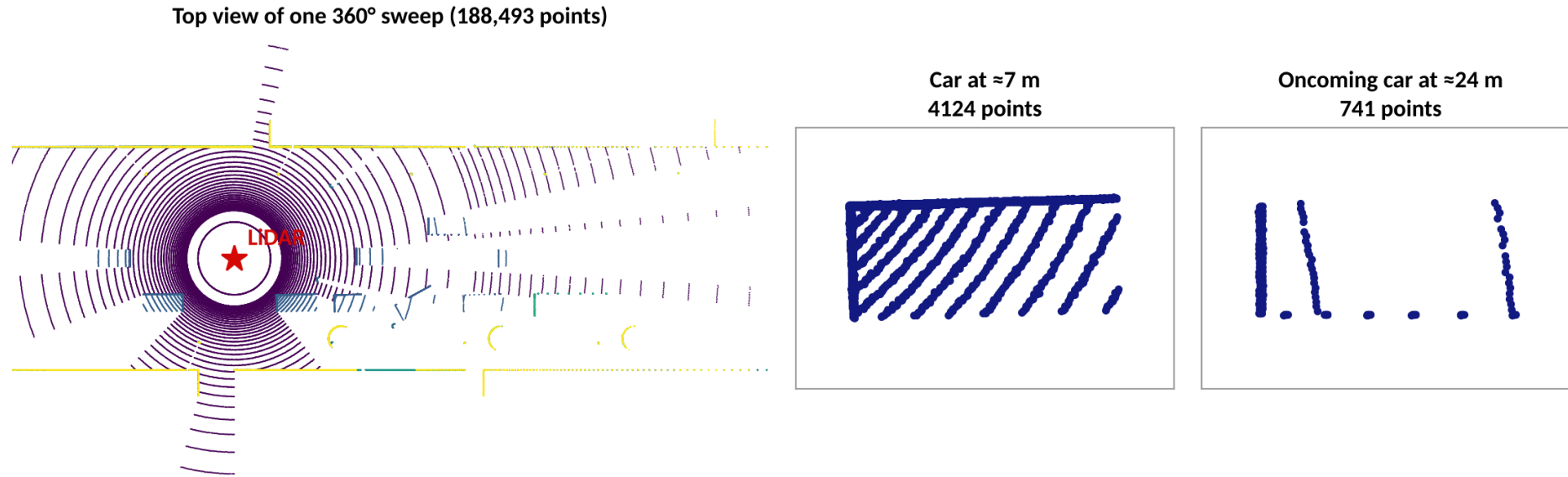
- Several LiDARs per vehicle multiply the rate (Waymo Open Dataset records 5 LiDARs)
- Perception must run at the sweep rate (≥ 10 Hz): ≈ 100 ms budget end-to-end
- Hence the interest in compact representations (range image, pillars) and sparse operators
- Logging, upload, and labeling costs scale with data volume, not just compute

Part 2: Point Cloud Data



Properties | coordinate frames | motion distortion | representations | voxel sparsity | datasets

What Is a Point Cloud?

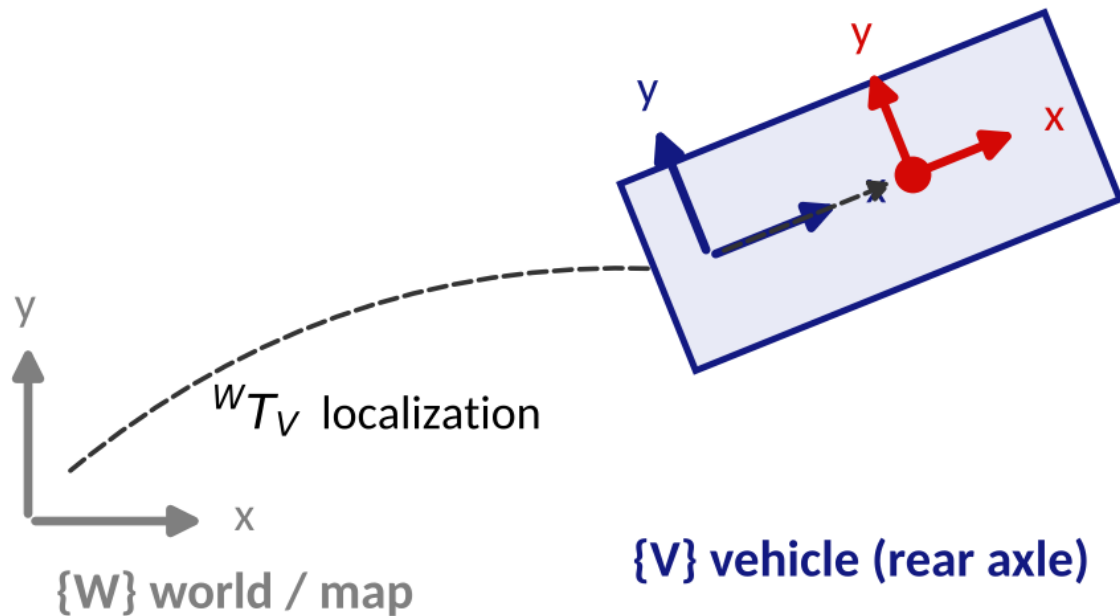


- An **unordered set** of points $\{(x, y, z, \text{intensity}, t)\}$: no grid, no neighbor information
- **Density falls with range**: $\approx 4,100$ points on a car at 7 m, only ≈ 740 on an oncoming car at 24 m
- **Occlusion**: only surfaces facing the sensor are sampled (the typical 'L-shape' of a car)
- This synthetic 128-channel sweep has 188,493 returns; real sensors give 10^5 – 10^6 per sweep

Coordinate Frames and Calibration

${}^V T_L$: extrinsic calibration
(fixed mounting pose)

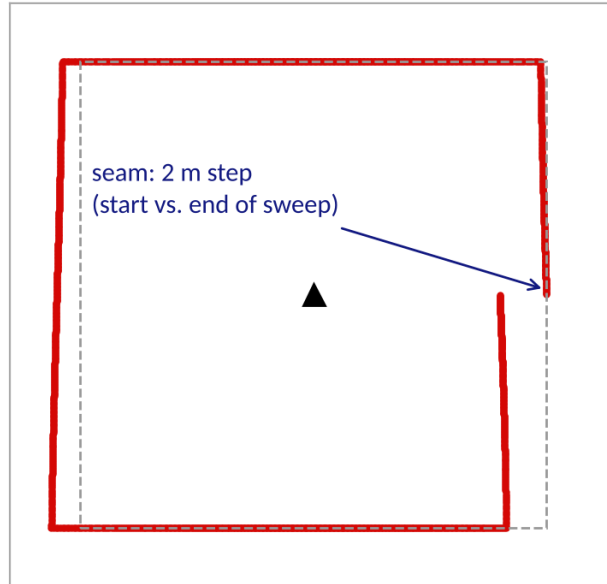
{L} LiDAR (roof)



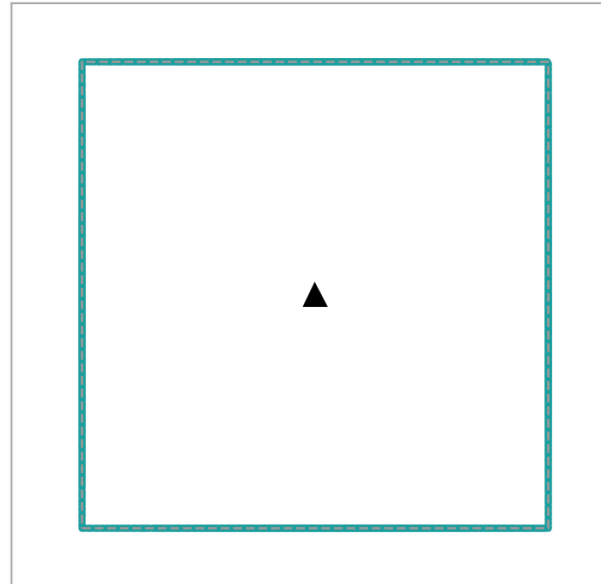
- Points are measured in the **LiDAR frame {L}**; planning needs the vehicle {V} or world {W} frame
- $\mathbf{p}_V = \mathbf{R} \cdot \mathbf{p}_L + \mathbf{t}$ (extrinsic calibration, fixed)
- $\mathbf{p}_W = {}^W T_V \cdot {}^V T_L \cdot \mathbf{p}_L$ (localization from GNSS / IMU / SLAM)
- Camera fusion: $\lambda [\mathbf{u} \ \mathbf{v} \ 1]^T = \mathbf{K} (\mathbf{R}_c \mathbf{p}_L + \mathbf{t}_c)$
- A 0.5° rotation error shifts a point by ≈ 0.9 m at 100 m

Motion Distortion and Deskewing

Raw sweep: every point placed in the sensor frame at its own time



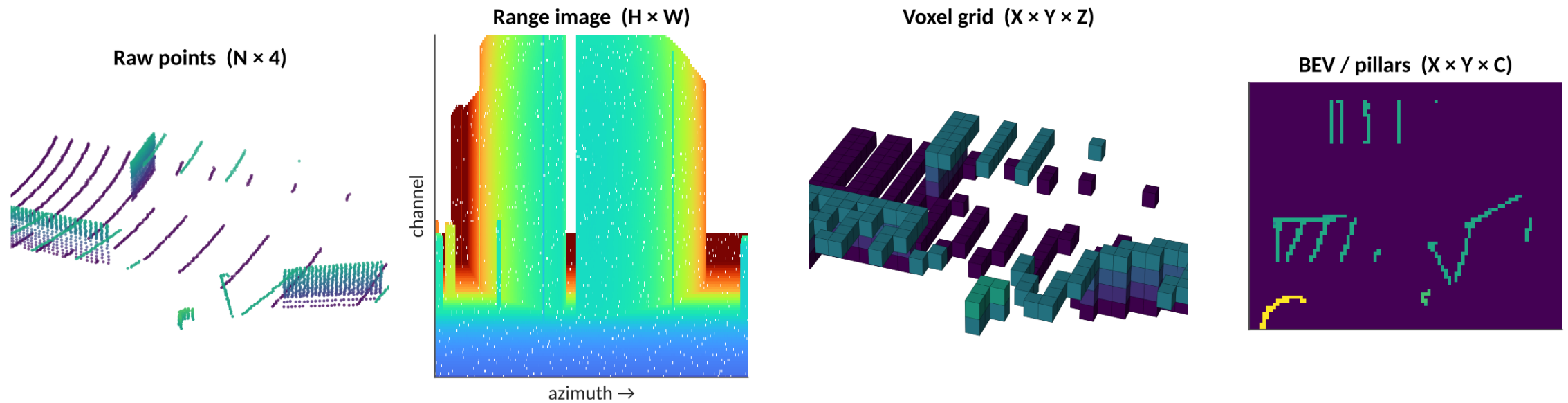
After ego-motion compensation (deskew)



Synthetic example: square room, ego speed 20 m/s along +x, sweep starts at azimuth 0°. The 2 m step appears where the sweep begins and ends.

- A spinning LiDAR needs ≈ 100 ms per 360° sweep (10 Hz)
- At 20 m/s the car moves **2 m** during one sweep
- Each point carries its own timestamp; first and last points see different poses
- **Deskew**: transform every point with the pose at its timestamp (IMU / odometry interpolation)
- Ignoring it blurs walls and creates a seam at the start/end azimuth

Four Ways to Represent a Sweep



	Points	Range image	Voxels	Pillars / BEV
Structure	irregular set ($N \times 4$)	2D grid in (θ, ϕ)	regular 3D grid	2D map from above
Strength	no information loss	dense, fast 2D CNNs	3D locality, convolutions	cheap, scale-consistent
Weakness	needs set-based ops	scale varies, occlusion	mostly empty, memory	height is squashed

Voxelization and Sparsity

- Voxel grid: cut space into cubes of size v (0.1–0.4 m) and group points per cube
- Region $80\text{ m} \times 80\text{ m} \times 4\text{ m}$ at $v = 0.1\text{ m} \rightarrow 800 \times 800 \times 40 = \mathbf{25.6\text{ M voxels}}$
- One sweep has $\approx 0.19\text{ M}$ points $\rightarrow < \mathbf{1\%}$ of voxels are non-empty
- Dense 3D convolution spends $> 99\%$ of its work on empty space
- Remedies: **sparse convolution** (SECOND), **pillars** (no z split), **range / BEV** projection
- Per-voxel features: mean (x, y, z), intensity, point count

25.6 M

voxels in an $80 \times 80 \times 4\text{ m}$ region ($v = 0.1\text{ m}$)

0.19 M

points in one 128-channel sweep

< 1 %

of voxels are occupied

Open LiDAR Datasets

Dataset	Year	LiDAR	Scale	Notes
KITTI	2012	Velodyne HDL-64E, 10 Hz	7,481 train / 7,518 test frames	Cars, pedestrians, cyclists; boxes in camera FOV
nuScenes	2019	32-beam, 20 Hz (+ 6 cameras, 5 radars)	1,000 scenes × 20 s; 1.4 M boxes; 23 classes	360° annotations; Boston and Singapore
Waymo Open	2019	5 LiDARs (1 mid-range, 4 short-range)	1,150 scenes × 20 s (perception set)	High-density data; separate motion dataset
SemanticKITTI	2019	KITTI odometry sequences	Point-wise labels	Semantic segmentation benchmark

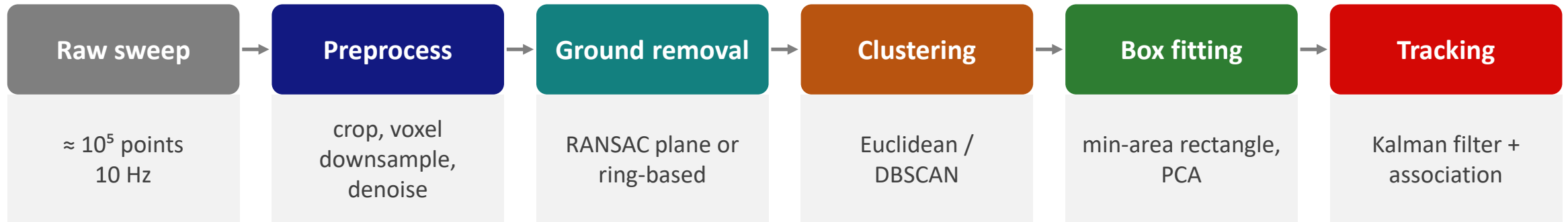
Tip: start with KITTI for debugging, then switch to nuScenes or Waymo Open for 360° evaluation.

Part 3: Classical Point Cloud Perception



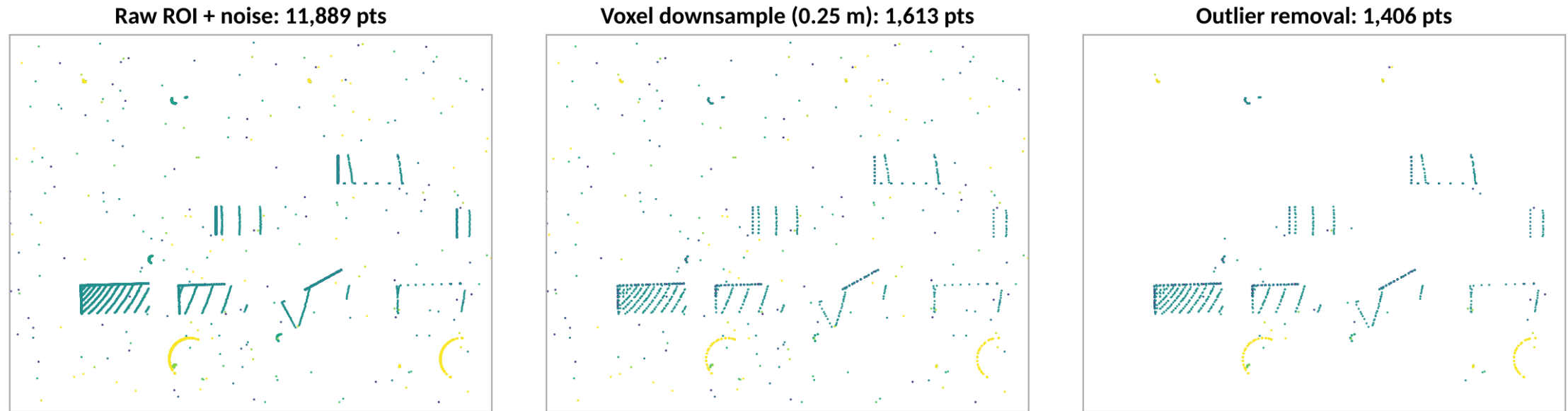
Pipeline | preprocessing | ground removal | clustering | boxes | ICP | tracking | limits

The Classical Perception Pipeline



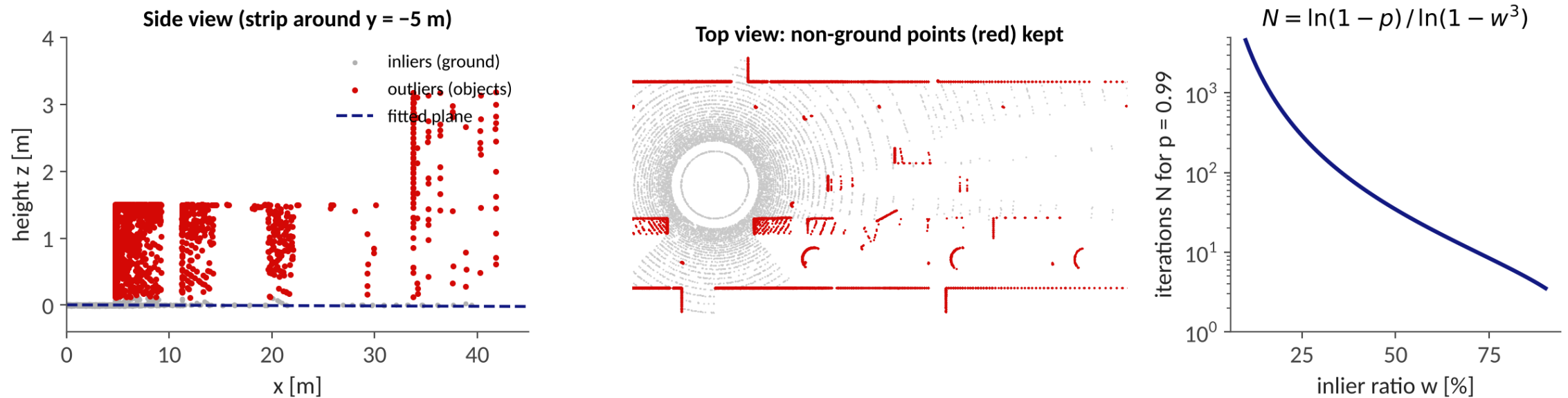
- **No training data**: each stage has a few interpretable parameters (ϵ , thresholds, voxel size)
- Fast and explainable; runs on a CPU; easy to debug visually
- Still used as a **baseline, a safety fallback, and a pre-labeling tool**
- Output so far: a list of oriented 3D boxes, but **no class** and no understanding of context
- Open-source toolkits: Point Cloud Library (PCL) and Open3D

Step 1: Preprocessing



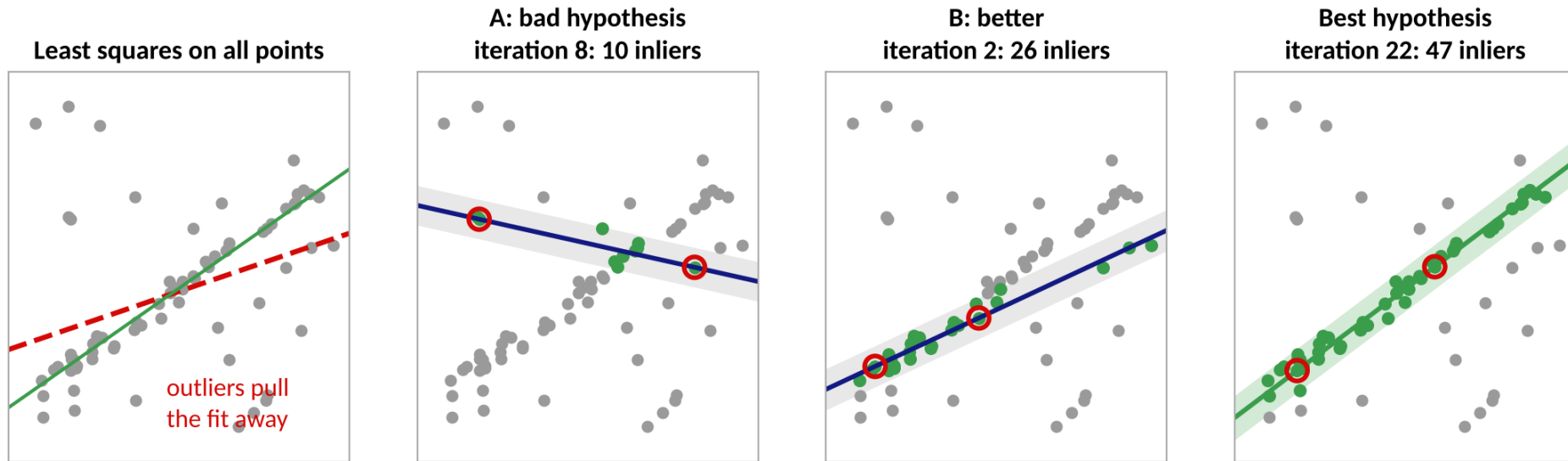
- **Crop** to a region of interest (e.g., $|y| < 12$ m, z within the vehicle's height band)
- **Voxel-grid downsampling**: one centroid per 0.25 m voxel, 11,889 $\rightarrow$ 1,613 points here (–86 %)
- **Statistical outlier removal**: drop points whose mean neighbor distance exceeds $\mu + \sigma$ ($k = 8$), removing most injected noise
- Parameter trade-off: coarser voxels are faster but erase small objects such as pedestrians and poles

Step 2: Ground Removal with RANSAC



- **RANSAC:** sample 3 low points $\rightarrow$ plane $\rightarrow$ count points within 0.12 m $\rightarrow$ keep the plane with most inliers
- Sample only from low points ($z < 0.5$ m) so the fitted plane is the road, not a wall
- Iterations $N = \ln(1-p) / \ln(1-w^3)$: inlier ratio $w = 0.5$, $p = 0.99 \rightarrow N \approx 35$
- Real roads curve and slope: use zone-wise / ring-based fitting (e.g., Patchwork-style) instead of one global plane

RANSAC: Fit Despite Outliers



- **Problem:** least squares uses every point, so outliers (cars, walls, poles) bend the fit
- **Idea:** guess a model from a minimal random sample, let all points vote, keep the model with the most inliers
- A line needs **2 points**; a plane (ground) needs **3 points**; each guess is cheap to fit and to score
- Inliers = points within threshold τ of the model (the shaded band); the last step refits on the inliers

RANSAC: Steps and Iterations

```
best = {}
repeat N times:
    S = random sample of s points
    model = fit(S)           # s=3: a plane
    I = {p : dist(p, model) < tau}
    if |I| > |best|: best = I
model = least_squares_fit(best) # refit
return model, best
```

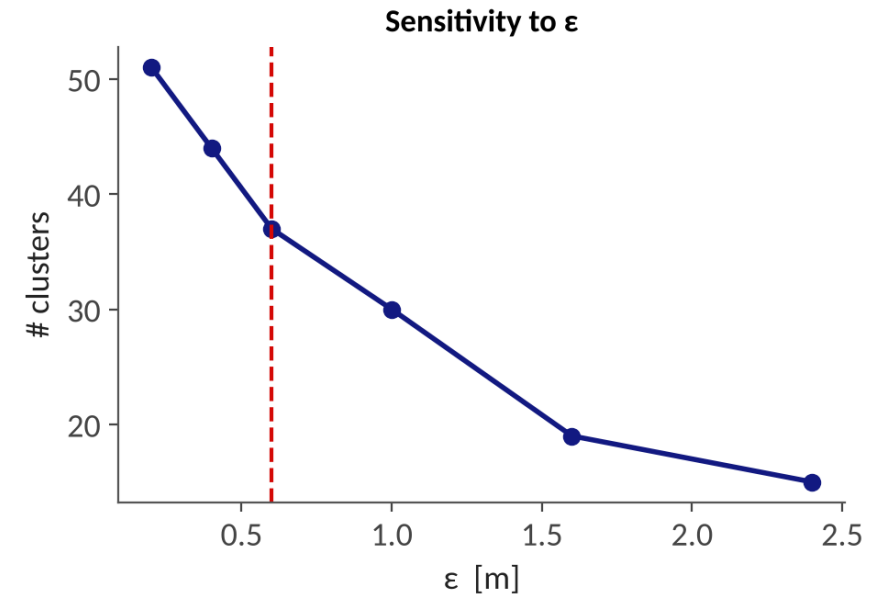
Inlier ratio w	N (line, s = 2)	N (plane, s = 3)
90%	3	4
70%	7	11
50%	17	35
30%	49	169
10%	459	4,603

$$N = \ln(1 - p) / \ln(1 - w^s), \text{ confidence } p = 0.99$$

- τ (threshold): about 3σ of range noise plus road roughness; 0.1–0.2 m for ground planes
- Low inlier ratio or large s makes N explode: use minimal samples and restrict candidates ($z < 0.5$ m)
- Random, not deterministic: fix the seed in tests, stop early once enough inliers are found
- Finds one model per run; to find several (e.g., two walls) remove the inliers and repeat

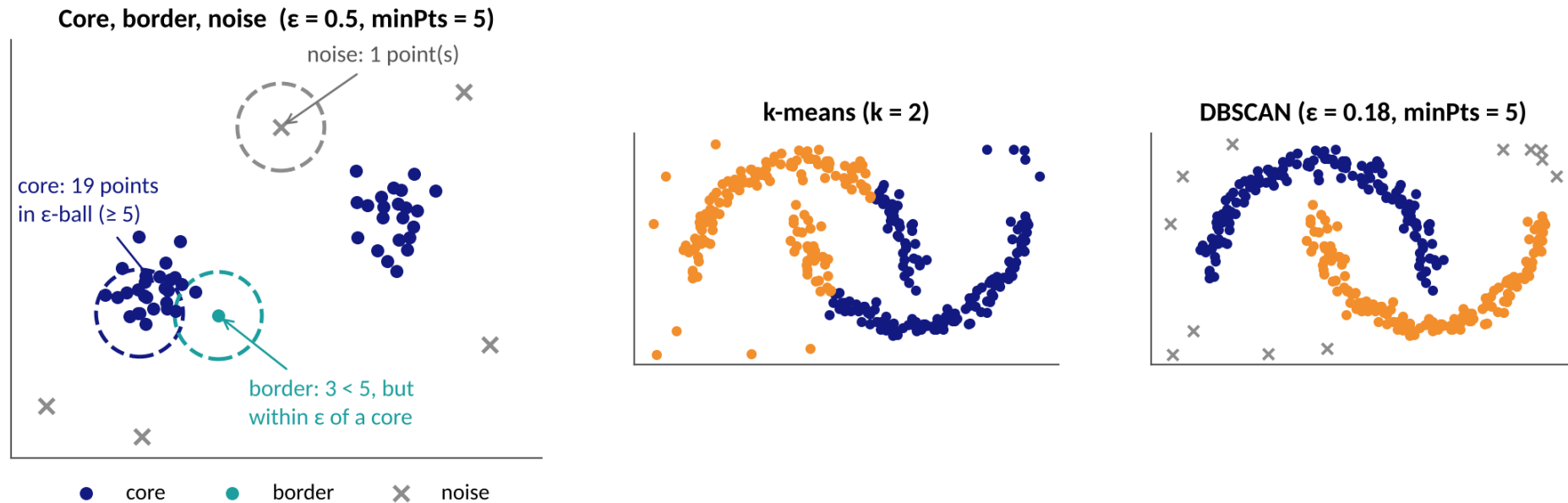
Step 3: Clustering Objects

DBSCAN ($\epsilon = 0.6$ m, minPts = 6): 37 clusters (grey = noise)



- **Euclidean clustering / DBSCAN:** connect points closer than ϵ , grow clusters with a KD-tree; points in sparse regions become noise
- Here $\epsilon = 0.6$ m and minPts = 6 give **37 clusters** (cars, pedestrians, poles, trees, wall pieces)
- ϵ is a trade-off: too small splits objects, too large merges neighbors (right plot)
- Far objects are sparser (slide 9): use a range-dependent $\epsilon(R)$

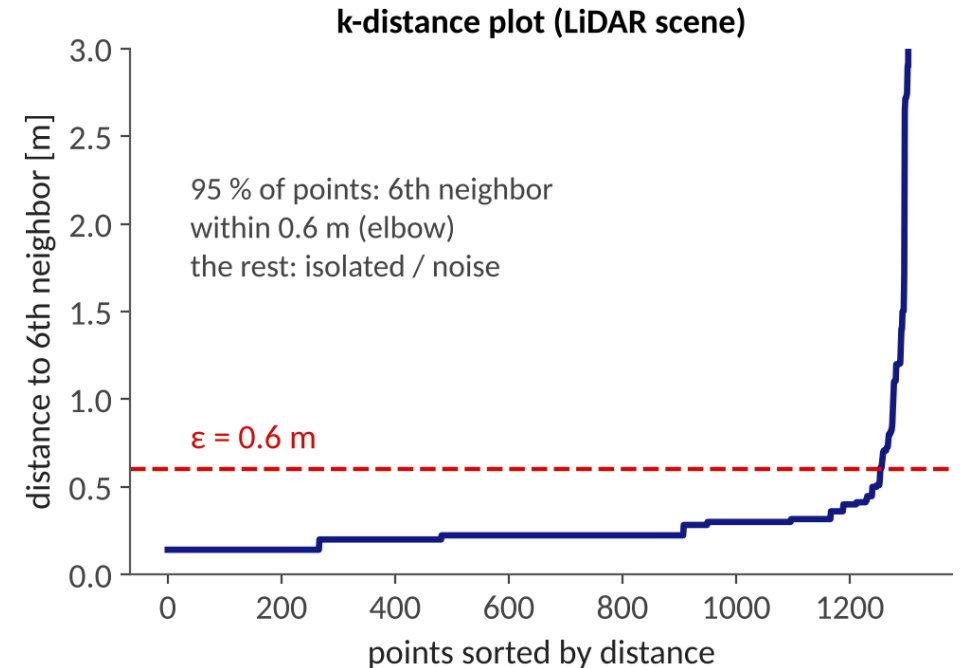
DBSCAN: Core, Border, Noise



- **Core** point: at least minPts points within distance ϵ ; **border**: fewer, but within ϵ of a core; **noise**: neither
- A cluster grows by chaining overlapping ϵ -balls of core points, so it can take **any shape**
- No need to choose the number of clusters, and sparse noise is rejected (k-means assigns every point)

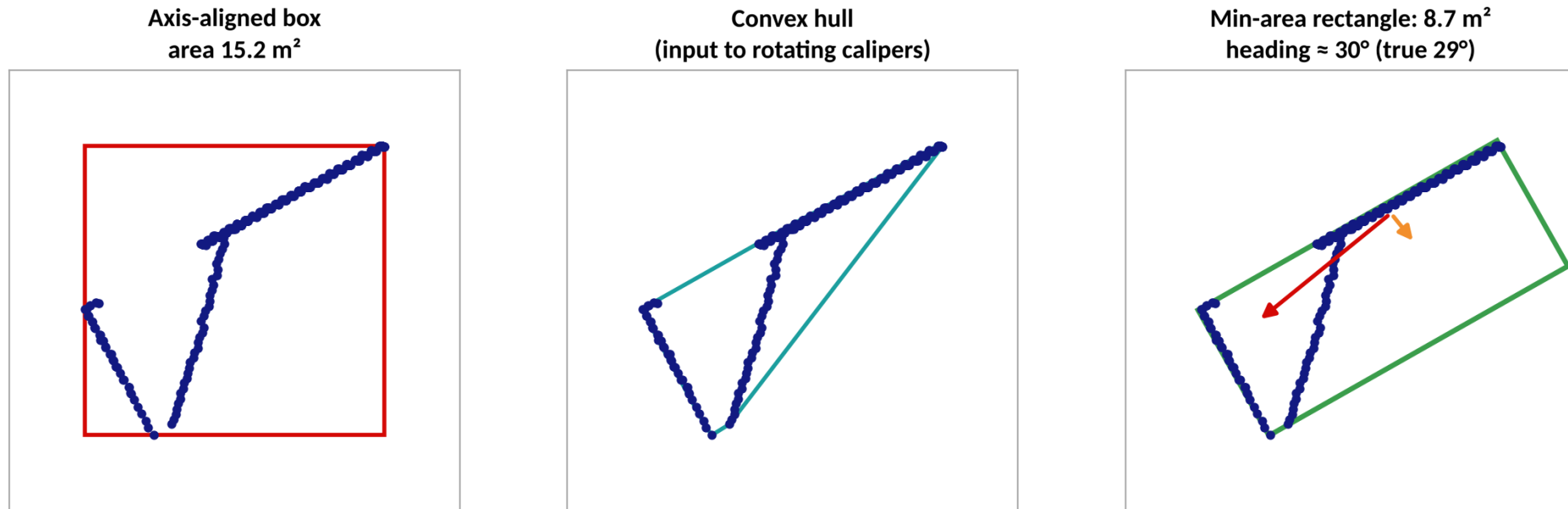
DBSCAN: Choosing ϵ and minPts

```
for each unvisited point p:
    mark p visited; N = neighbors(p,  $\epsilon$ )
    if |N| < minPts: label p noise; continue
    new cluster C; add p to C
    for each q in N:                # queue
        if q unvisited:
            mark q visited
            N' = neighbors(q,  $\epsilon$ )
            if |N'| >= minPts: N = N + N'
    if q in no cluster: add q to C
```



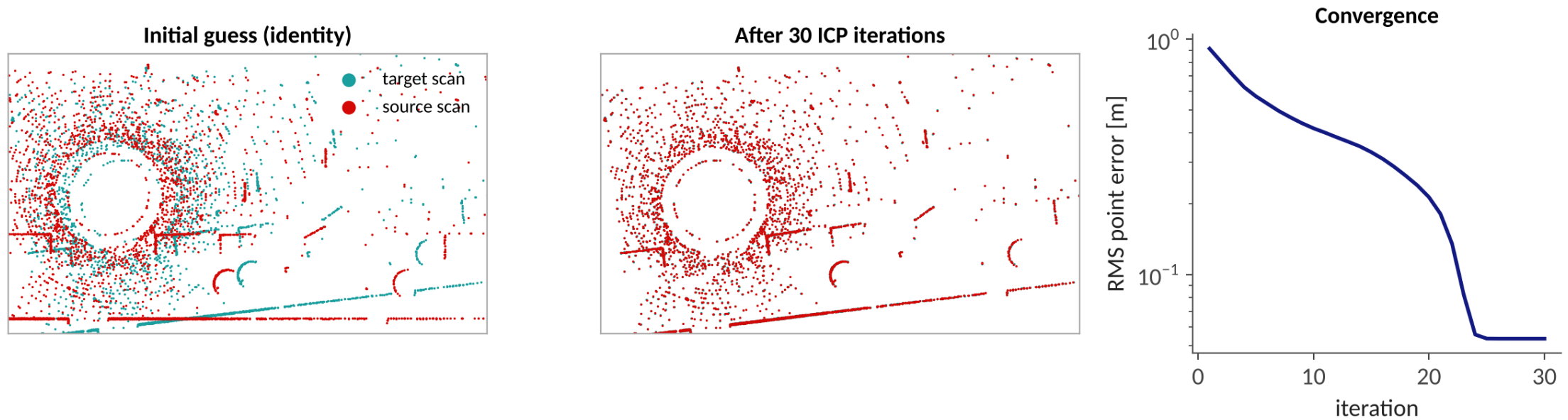
- **minPts** $\approx$ 4–6 for 2D (top-view) clustering; ϵ at the elbow of the k-distance plot (here $\approx$ 0.6 m)
- KD-tree neighbor queries give $O(n \log n)$: downsample (0.1 m voxels) before clustering
- Point spacing $s = R \cdot \tan \Delta\theta$ grows with range (0.07 m at 20 m, 0.21 m at 60 m for 0.2°): one ϵ does not fit near and far
- Remedies: range-adaptive $\epsilon(R)$, clustering on the range image, or HDBSCAN

Step 4: Fitting an Oriented Box



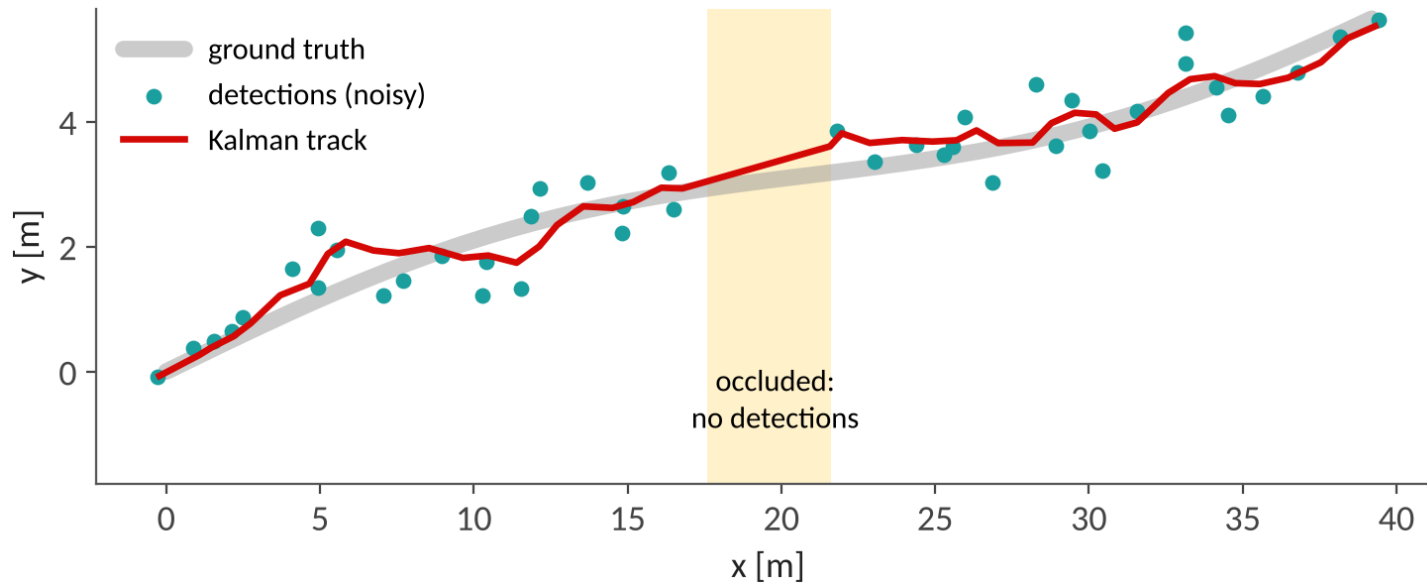
- An axis-aligned box over-estimates a rotated car: **15.2 m² vs. 8.7 m²** for the min-area rectangle
- **Min-area rectangle:** try headings 0–90°, keep the smallest enclosing rectangle (or rotating calipers on the convex hull)
- **PCA** gives a quick heading; **L-shape fitting** uses the two visible faces of a car
- Recovered heading ≈ 30° vs. true 29°; box height from the z-extent, center z from the ground plane

Registration with ICP



- Align scan B to scan A: minimize $\sum \|R p_i + t - q_i\|^2$ over matched pairs
- Loop: nearest-neighbor matching (KD-tree) $\rightarrow$ closed-form (R, t) by **SVD** $\rightarrow$ apply $\rightarrow$ repeat
- Needs a good initial guess (odometry): otherwise it converges to a local minimum
- Variants: point-to-plane, NDT, GICP; LOAM matches edge and plane features. Uses: odometry, mapping, localization
- Here (synthetic, 7° and 1.4 m offset): RMS error 0.91 m $\rightarrow$ 0.05 m in 30 iterations

Step 5: Tracking with a Kalman Filter



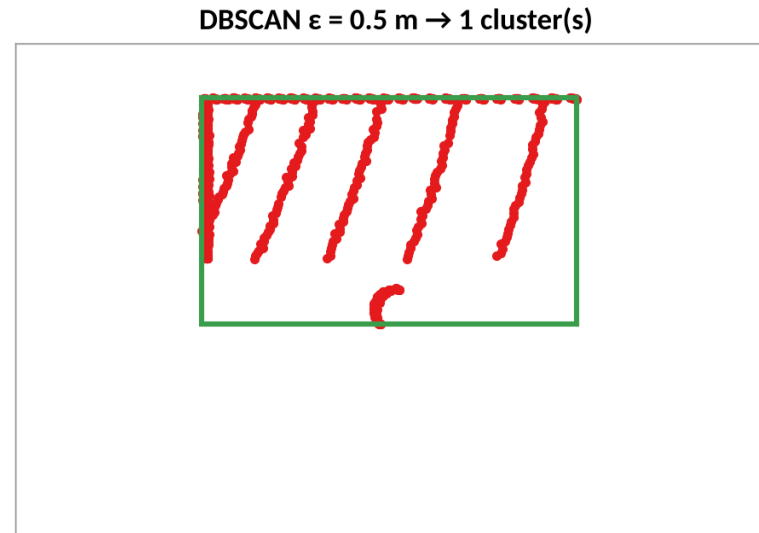
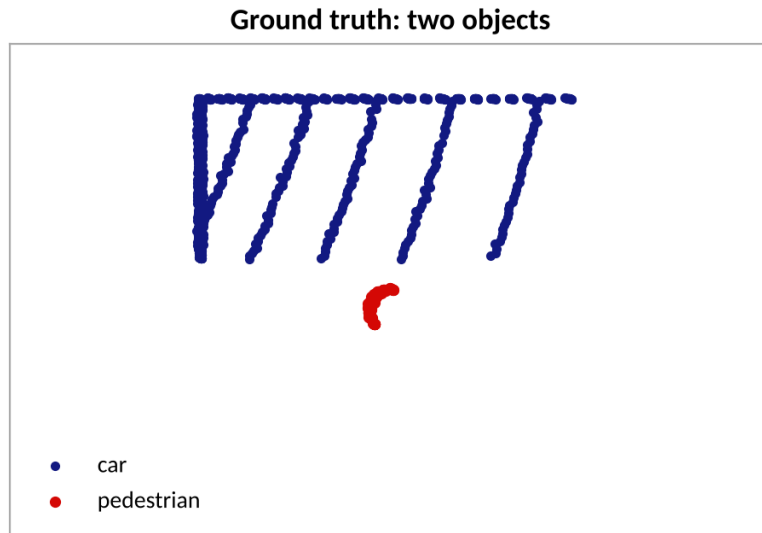
- **Associate** detections with tracks (nearest neighbor or Hungarian algorithm on distance / IoU)
- **Kalman filter**, state $[x, y, v_x, v_y]$: predict with constant velocity, update with the detection
- Prediction **carries the track through an occlusion** (here 0.6 s without detections)
- Track management: birth, confirmation, deletion

Predict: $\hat{x} = F x$, $P = F P F^T + Q$

Update: $K = P H^T (H P H^T + R)^{-1}$, $x = \hat{x} + K (z - H \hat{x})$

Here: RMS error 0.66 m (raw detections) $\rightarrow$ 0.37 m (filtered)

Where the Classical Pipeline Breaks

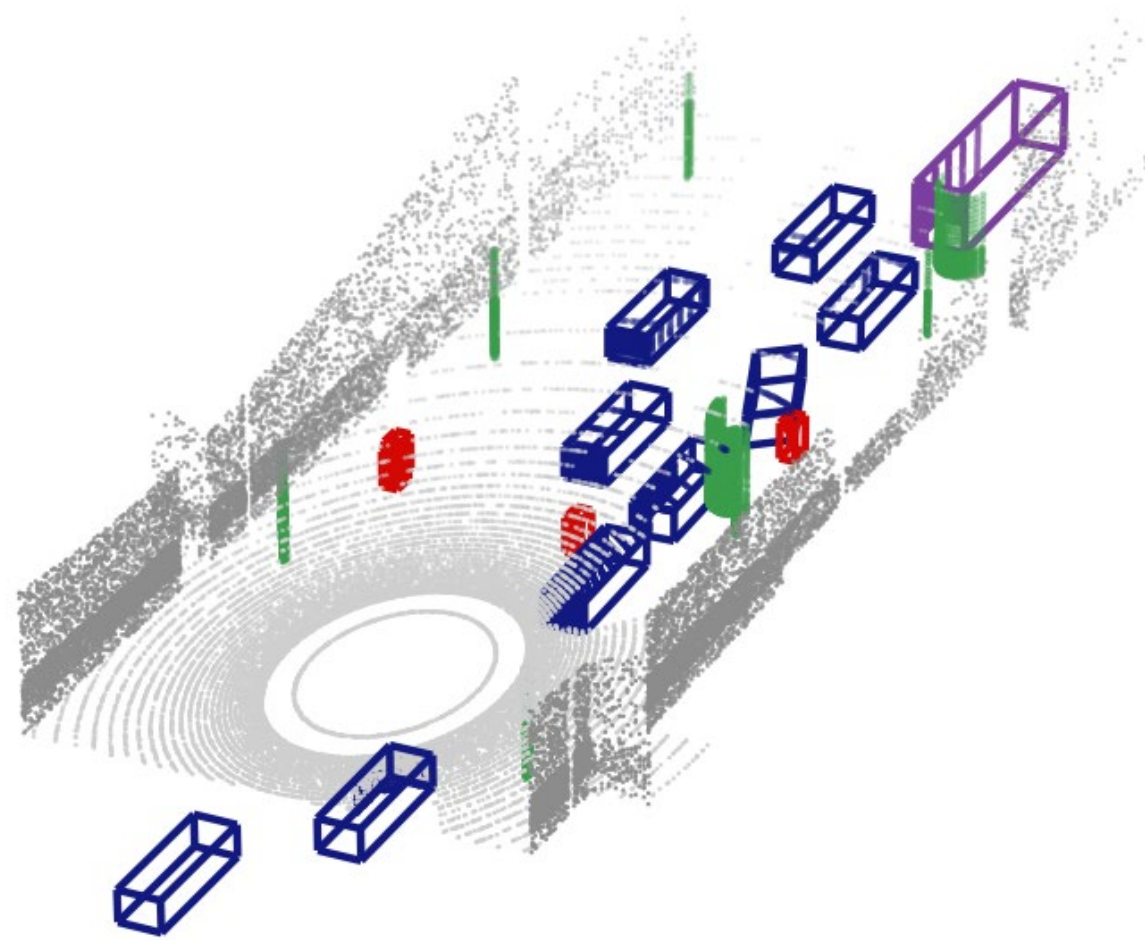


- **Under-segmentation:** pedestrian 0.3 m from a car $\rightarrow$ one cluster (2,077 + 109 points)
- **Over-segmentation:** far objects split into fragments
- **No semantics:** cannot tell a car from a bush
- Thresholds need re-tuning per scene and sensor

$\rightarrow$ Learned detectors use **context and appearance** to separate and classify objects

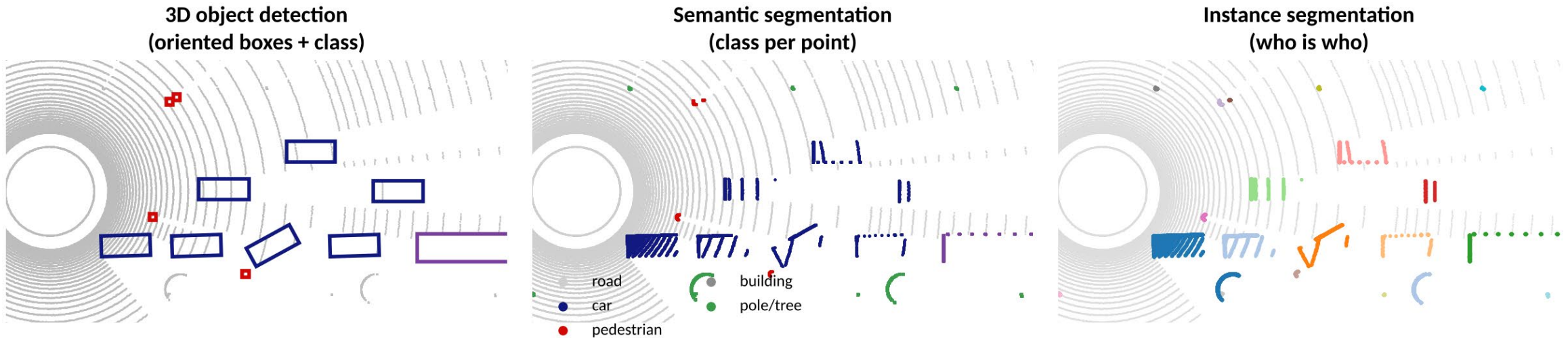
$\rightarrow$ Classical stages remain valuable as **fallbacks**, for **labeling tools**, and for **sanity checks** on network outputs

Part 4: Deep Learning Perception



Tasks | PointNet | voxels | pillars | BEV | metrics | fusion | Waymo | latency and robustness

Perception Tasks on Point Clouds



- **3D object detection** (our focus): oriented box (x, y, z, l, w, h, θ) + class + confidence for every object
- **Semantic segmentation**: a class label per point (e.g., SemanticKITTI); **instance segmentation** adds object identity
- Tracking and motion forecasting consume the detections over time
- Supervision: human-labeled 3D boxes or per-point labels; auto-labeling is increasingly used

Why Point Clouds Are Hard to Learn

Unordered

N points can be listed in $N!$ orders; the output must not change

Remedy: symmetric functions (max-pool), as in PointNet

Sparse and non-uniform

Density falls $\approx 1/R^2$; far objects have only a handful of points

Remedy: sparse convolution, learned features, larger receptive fields

Large scale

$\approx 10^5$ points per sweep and a ≈ 100 ms budget at 10 Hz

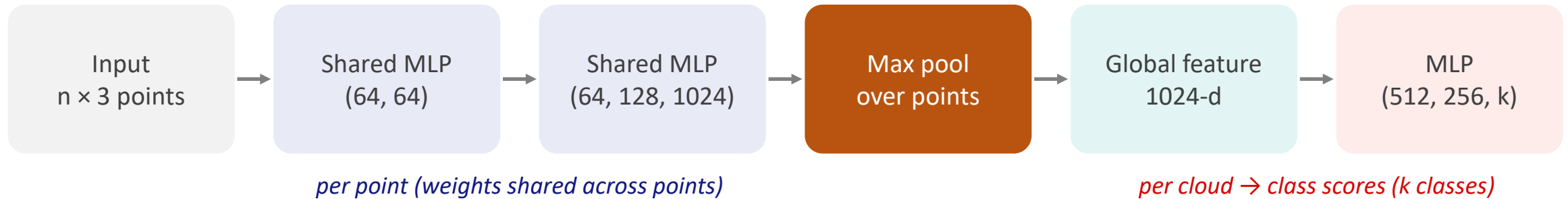
Remedy: voxel / pillar grids, range images, efficient 2D backbones

No regular grid

No neighbors and no pixels: standard CNNs do not apply directly

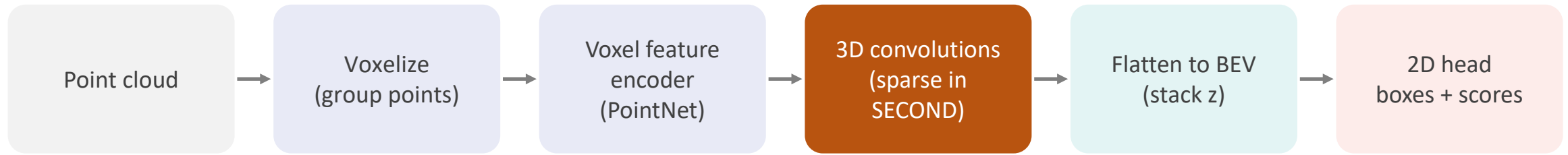
Remedy: voxelize or project to a grid, or use point-based set operators

PointNet: Learning on Raw Points



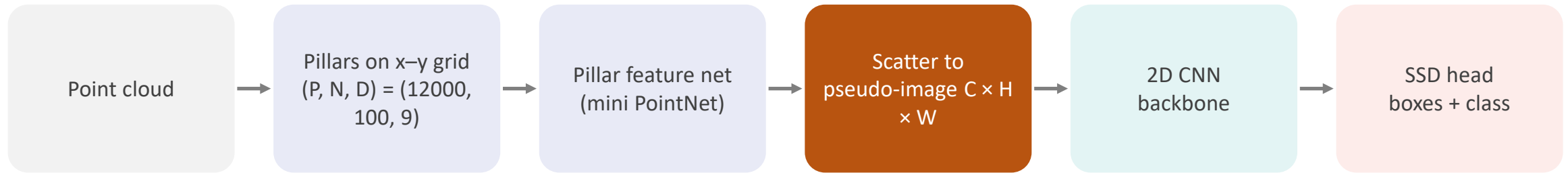
- $f(\{x_1 \dots x_n\}) \approx \gamma(\text{MAX}_i h(x_i))$: h is shared per point and max-pool is symmetric, so the output is **order-invariant**
- A small **T-Net** predicts a transform that aligns the input (pose normalization)
- For segmentation, concatenate the global feature with each per-point feature
- Limitation: no local neighborhoods → **PointNet++** adds hierarchical sampling and grouping
- Cost: neighbor search and sampling are slow on 10^5 points per sweep, so detectors moved to grids

Voxel Detectors: VoxelNet and SECOND



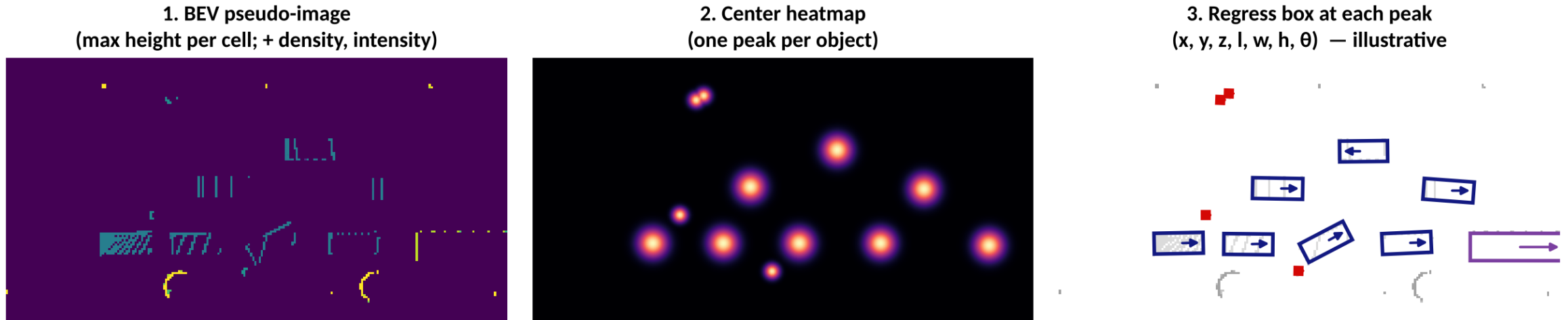
- **VoxelNet** (2018): first end-to-end voxel detector; per-voxel PointNet features + dense 3D convolution, but slow and memory-hungry
- **SECOND** (2018): **sparse 3D convolution** only touches non-empty voxels (< 1 % of the grid, slide 21) → much faster
- SECOND also popularized **ground-truth sampling**: paste labeled objects from other scenes into the current one
- Strong accuracy, but needs custom sparse-conv kernels → harder to deploy on embedded GPUs

PointPillars: Pillars + 2D CNN



- Pillars drop the z split → **only 2D convolutions**: simple to deploy (e.g., TensorRT) and fast, **62 Hz** in the paper
- Point decoration: x, y, z, intensity + offsets to pillar mean (3) and pillar center (2) → D = 9
- Fixed buffers: $\leq P = 12,000$ non-empty pillars and $\leq N = 100$ points per pillar (extra points are sampled away)
- Trade-off: vertical structure inside a pillar is only captured by learned features
- A good default for real-time systems and a common baseline in toolboxes such as OpenPCDet

BEV Features and Center-Based Detection



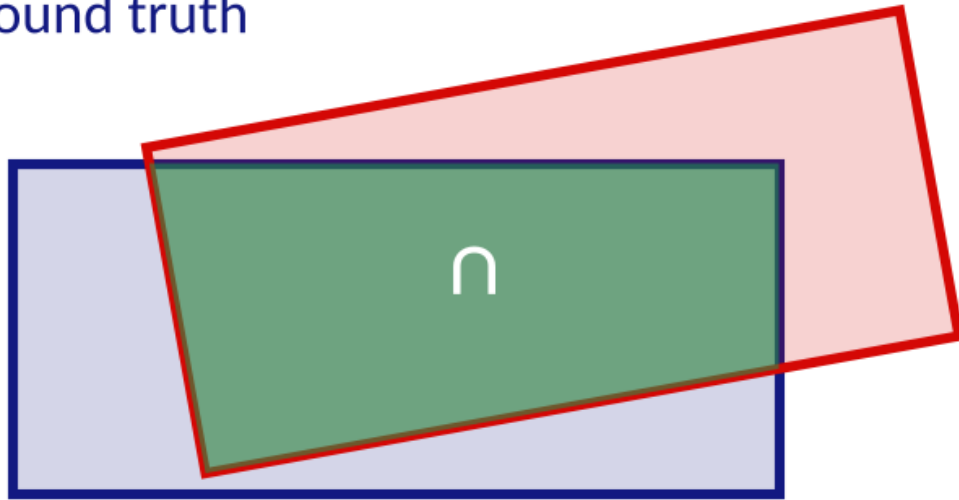
- **BEV** (bird's-eye view): objects do not overlap, size does not change with distance, and it matches the planning space
- **CenterPoint**: predict a heatmap peak at each object center, then regress size, height, heading (sin, cos), and velocity
- Anchor-free: no anchor boxes; tracking by matching center offsets between frames
- Waymo (ISSW 2024): BEV aggregation improves generalization and supports plan validation

Comparing Detector Families

Family	Representation	Examples	Strengths	Weaknesses
Point-based	raw points	PointNet++, PointRCNN	fine geometry	slow neighbor search at scale
Range-view	range image	LaserNet, RangeNet++	fast 2D CNNs	scale varies; occlusion
Voxel	3D grid + sparse conv	VoxelNet, SECOND	accurate	memory; custom ops
Pillar / BEV	2D pseudo-image	PointPillars, CenterPoint	fast, easy to deploy	height is squashed
Point-voxel	hybrid	PV-RCNN	fine + coarse features	complex two-stage pipeline
Sparse transformer	sparse windows	SWFormer (Waymo)	long-range context	newer, compute-heavy

How Detections Are Scored

ground truth

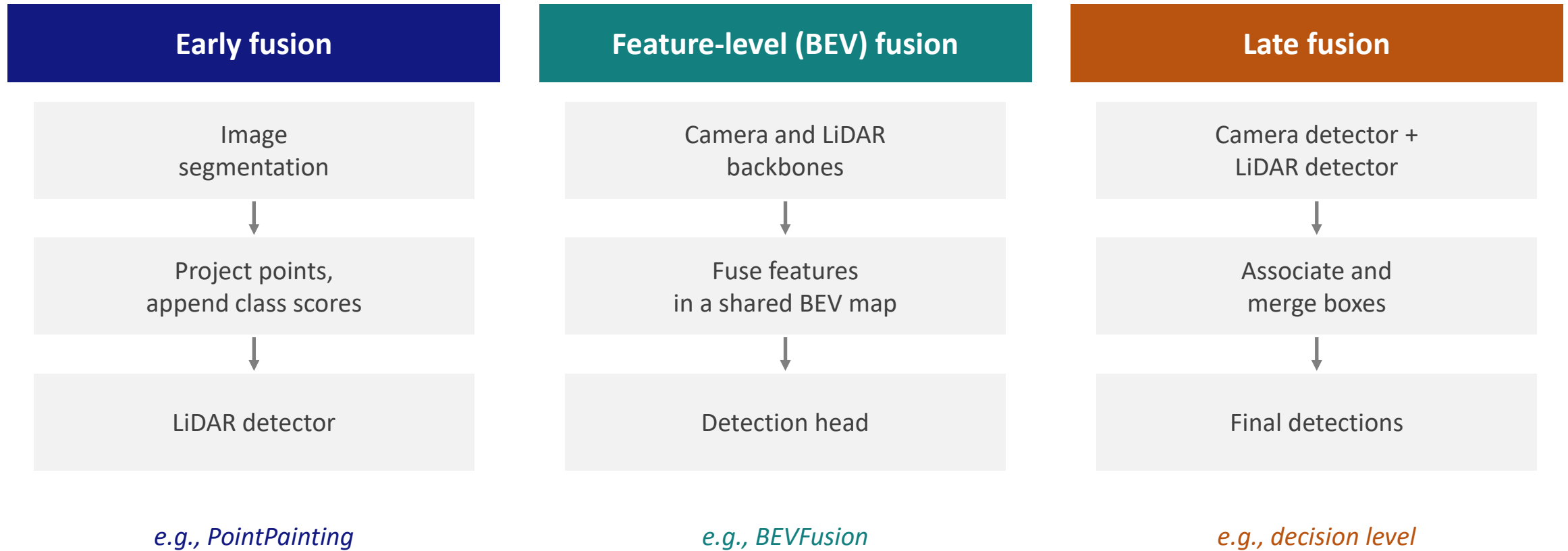


prediction

Example: 4.4 m × 1.9 m boxes, 0.9 m shift and 10° heading error → BEV IoU = 0.45

- **IoU** = $|A \cap B| / |A \cup B|$
- **KITTI**: true positive if IoU ≥ 0.7 (car), ≥ 0.5 (pedestrian, cyclist); AP over the precision–recall curve
- **nuScenes**: match by center distance (0.5, 1, 2, 4 m) → mAP; **NDS** adds box, attribute, velocity errors
- **Waymo**: AP and heading-weighted **APH**, with L1 / L2 difficulty levels
- Small objects: a 0.3 m shift can cut IoU sharply

LiDAR–Camera Fusion



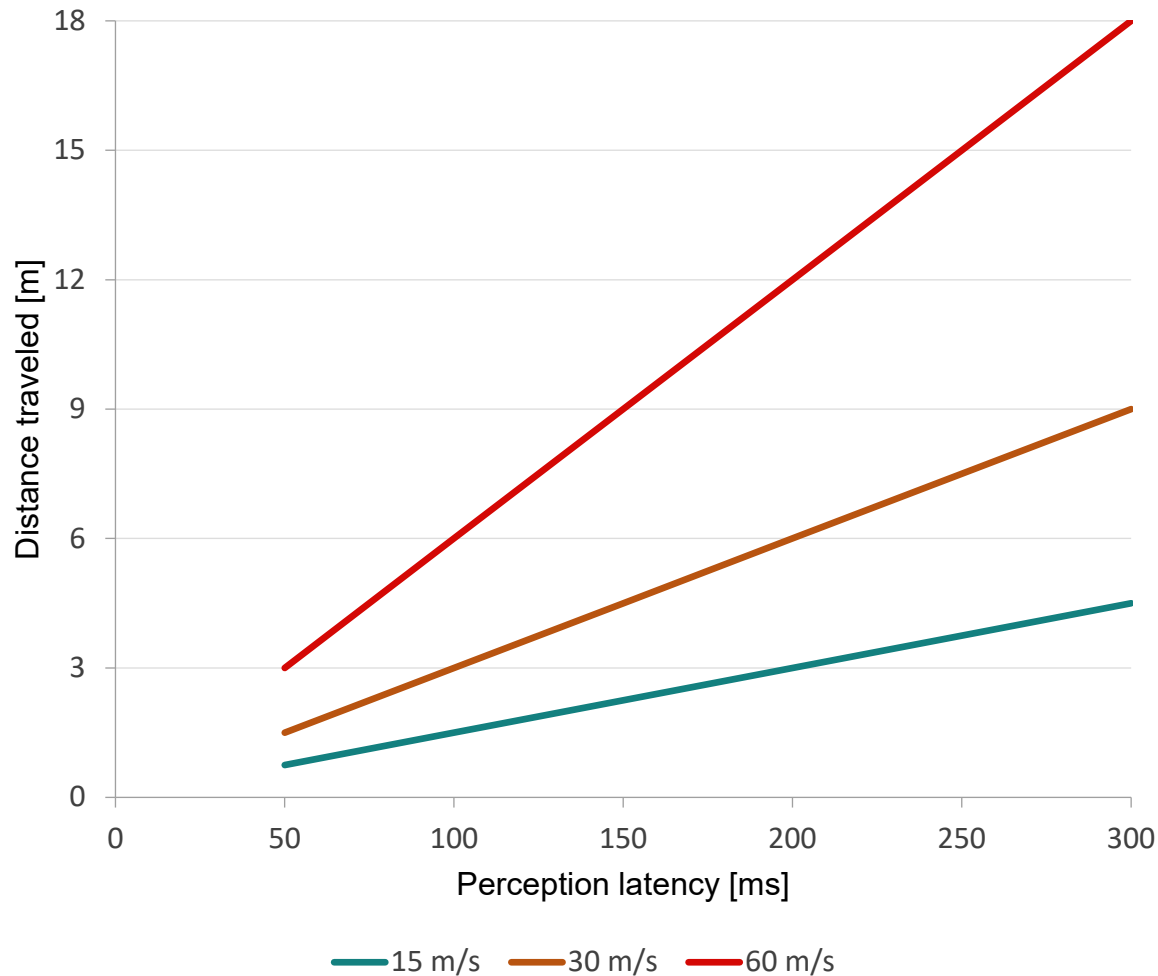
Why fuse: LiDAR = geometry, camera = color and texture. Requirements: precise extrinsics and time synchronization.

Case Study: Waymo's LiDAR Perception



- Processes $O(10)$ sensors with **hundreds of millions of readings per second**; decisions at ≥ 10 Hz
- **SWFormer** (sparse window transformer): dense convolution grows quadratically with range, sparse convolution is awkward on TPUs
- **Waymo Open Dataset** is a public benchmark with camera + LiDAR and rich 3D labels
- **Offboard auto-labeling** with object-centric pipelines produces labels at scale
- **Vision-language models** transfer concepts such as 'bulldozer' or 'firetruck' to 3D detectors without 3D labels
- Reported: detection beyond 200 m at night on freeways

Latency and Robustness



- **Distance = speed × latency:** 100 ms at 60 m/s closing speed = 6 m
- Worst-case latency matters for safety, not just the average
- Levers: pillars / 2D CNNs, sparse ops, ROI cropping, FP16 / INT8 (TensorRT)
- **Adversarial LiDAR attacks:** spoofed pulses inject points; Cao et al. (2019) made a LiDAR detector (Apollo) see a non-existent obstacle
- Defenses: sensor fusion, consistency checks, adversarial training

Lab and Discussion

Hands-on lab

1. Load a KITTI or nuScenes sweep with **Open3D**
2. Implement **RANSAC ground removal + DBSCAN** and draw oriented boxes
3. Run a pretrained **PointPillars** (OpenPCDet or MMDetection3D)
4. Compare both on cars: precision and recall at IoU 0.5
5. Plot detections vs. range: where does each method fail?

Discussion questions

1. Why is a pedestrian at 100 m hard for a 0.2° LiDAR?
2. When would classical clustering beat a neural network?
3. What goes wrong if the extrinsic calibration is off by 0.5° ?
4. Pillars or voxels: which would you deploy on a vehicle GPU, and why?
5. Which sensor failure in slide 14 is hardest to fix in software?

Summary and Further Resources

Key takeaways

- A LiDAR measures **time of flight**; spacing, weather, and reflectivity set what you see
- Point clouds are **unordered, sparse, range-dependent** and need correct frames and deskewing
- Classical pipelines (ground → clusters → boxes → tracks) are fast but **brittle**
- Deep detectors trade representation for speed: **points, voxels, pillars, BEV**
- Always evaluate with **IoU-based AP** and consider **latency and robustness**

Further resources

- **Khizbullin**: LiDAR Perception for ADAS and AD (GitHub; Modules 3–4, Google Slides)
- **Waymo**, Lidar and Autonomous Cars (ISSW 2024, PDF)
- **Univ. Tübingen**: Self-Driving Cars, L10.5 3D Object Detection
- **Univ. Utah**: CS7960 self-driving cars lecture (latency and safety)
- **OpenPCDet** and **MMDetection3D**: reference implementations

References

- [1] S. Verghese (Waymo), Lidar and Autonomous Cars, ISSW 2024. [imagesensors.org/Past Workshops/2024 ISSW/Presentations/R05-1.pdf](https://imagesensors.org/Past%20Workshops/2024%20ISSW/Presentations/R05-1.pdf)
- [2] A. Geiger, P. Lenz, R. Urtasun, KITTI, CVPR 2012.
- [3] H. Caesar et al., nuScenes, CVPR 2020.
- [4] P. Sun et al., Waymo Open Dataset, CVPR 2020.
- [5] J. Behley et al., SemanticKITTI, ICCV 2019.
- [6] M. Fischler, R. Bolles, RANSAC, Commun. ACM 1981.
- [7] M. Ester et al., DBSCAN, KDD 1996.
- [8] P. Besl, N. McKay, A method for registration of 3-D shapes, TPAMI 1992.
- [9] J. Zhang, S. Singh, LOAM, RSS 2014.
- [10] C. Qi et al., PointNet, CVPR 2017; PointNet++, NeurIPS 2017.
- [11] Y. Zhou, O. Tuzel, VoxelNet, CVPR 2018.
- [12] Y. Yan, Y. Mao, B. Li, SECOND, Sensors 2018.
- [13] A. Lang et al., PointPillars, CVPR 2019.
- [14] T. Yin, X. Zhou, P. Krähenbühl, CenterPoint, CVPR 2021.
- [15] S. Shi et al., PV-RCNN, CVPR 2020.
- [16] P. Sun et al., SWFormer, ECCV 2022.
- [17] S. Vora et al., PointPainting, CVPR 2020; Z. Liu et al., BEVFusion, ICRA 2023.
- [18] Y. Cao et al., Adversarial sensor attack on LiDAR-based perception, ACM CCS 2019.
- [19] D. Khizbullin, LiDAR Perception for ADAS and AD, github.com/dmitrii-khizbullin/lidar_course
- [20] Univ. Tübingen Self-Driving Cars (A. Geiger); Univ. Utah CS7960 self-driving cars lecture.